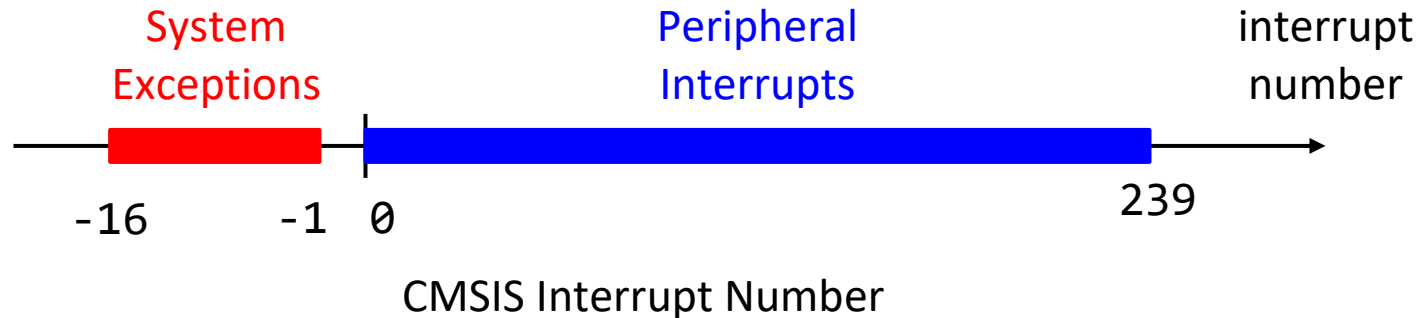


CSE331: Microprocessor Interfacing and Embedded Systems

Lecture#11: Interrupt Part#2 (Chapter#11)

Interrupt Number

- ❓ Cortex-M supports up to 256 interrupts.



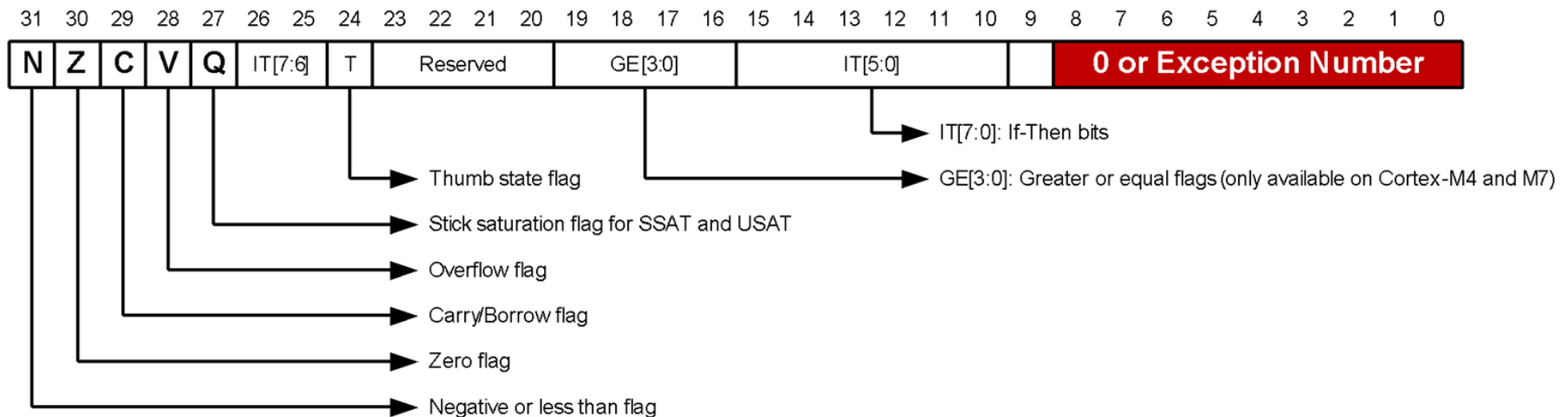
- ❓ First 16 are system exceptions
 - ❓ CMSIS defines their interrupt numbers as negative
 - ❓ Defined by ARM core
- ❓ The rest 240 are peripheral interrupts
 - ❓ Peripheral interrupt number starts with 0.
 - ❓ Defined by chip manufacturers.
 - ❓ NVIC= Nested Vector Interrupt Control
 - ❓ CMSIS= Common Microcontroller Software Interface Standard

Interrupt Number in CMSIS *vs* in PSR

Interrupt number for CMSIS functions

```
NVIC_DisableIRQ (IRQn);           // Disable interrupt  
NVIC_EnableIRQ (IRQn);           // Enable interrupt  
NVIC_ClearingPending (IRQn);     // clear pending status  
NVIC_SetPriority (IRQn, priority); // set priority level
```

Interrupt number in Program Status Register (PSR)



Interrupt Number in PSR = 16 + Interrupt Number (CMSIS)

CMSIS Interrupt Number

```

/***** Cortex-M4 System Exceptions *****/
NonMaskableInt_IRQn    = -14,    /* 2 Cortex-M4 Non Maskable Interrupt */
HardFault_IRQn         = -13,    /* 3 Cortex-M4 Hard Fault Interrupt */
MemoryManagement_IRQn = -12,    /* 4 Cortex-M4 Memory Management Interrupt */
BusFault_IRQn          = -11,    /* 5 Cortex-M4 Bus Fault Interrupt */
UsageFault_IRQn        = -10,    /* 6 Cortex-M4 Usage Fault Interrupt */
SVCall_IRQn            = -5,     /* 11 Cortex-M4 SV Call Interrupt */
DebugMonitor_IRQn      = -4,     /* 12 Cortex-M4 Debug Monitor Interrupt */
PendSV_IRQn            = -2,     /* 14 Cortex-M4 Pend SV Interrupt */
SysTick_IRQn           = -1,     /* 15 Cortex-M4 System Tick Interrupt */

/***** Peripheral Interrupt Numbers *****/
WWDG_IRQn              = 0,      /* Window WatchDog Interrupt */
PVD_PVM_IRQn           = 1,      /* PVD/PVM1,2,3,4 through EXTI Line detection Interrupts */
TAMP_STAMP_IRQn        = 2,      /* Tamper and TimeStamp interrupts through the EXTI line */
RTC_WKUP_IRQn          = 3,      /* RTC Wakeup interrupt through the EXTI line */
FLASH_IRQn             = 4,      /* FLASH global Interrupt */
RCC_IRQn               = 5,      /* RCC global Interrupt */
EXTI0_IRQn             = 6,      /* EXTI Line0 Interrupt */
...

```

**System
Exceptions**

**Peripheral
Interrupts**

[stm32l476xx.h](#)

Enable an Interrupt

❑ Enable a system exception

- ❑ Some are always enabled (cannot be disabled)
- ❑ No centralized registers for enabling/disabling
- ❑ Each are control by its corresponding components, such as SysTick module

❑ Enable a peripheral interrupt

- ❑ Centralized register arrays for enabling/disabling
- ❑ **ISER** registers for enabling
- ❑ **ICER** registers for disabling

Enabling Peripheral Interrupts

Interrupt Set Enable Register 0 (ISER0)

Enable Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Interrupt Number	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	I2C1_EV	TIM4	TIM3	TIM2	TIM11	TIM10	TIM9	LCD	EXTI9_5	COMP	DAC	USB_LP	USB_HP	ADC1	DMA1_CH7	DMA1_CH6	DMA1_CH5	DMA1_CH4	DMA1_CH3	DMA1_CH2	DMA1_CH1	EXTI4	EXTI3	EXTI2	EXTI1	EXTI0	RCC	FLASH	RTC_WKUP	TAMPER_STAMP	PVD	WWDG

Interrupt Set Enable Register 1 (ISER1)

Address of ISER1 = Address of ISER0 + 4

Enable Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Interrupt Number	44	43	42	41	40	39	38	37	36	35	34	33	32																			
	TIM7	TIM6	USB_FS_WKUP	RTC_Alarm	EXTI5_10	USART3	USART2	USART1	SPI2	SPI1	I2C2_ER	I2C2_EV	I2C1_ER																			

TIM7_IRQn = 44

NVIC->ISER[1] = 1 << 12; // Enable Timer 7 interrupt



Disabling Peripheral Interrupts

Interrupt Clear Enable Register 0 (ICER0)

Clear Enable Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Interrupt Number	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	I2C1_EV	TIM4	TIM3	TIM2	TIM11	TIM10	TIM9	LCD	EXTI9_5	COMP	DAC	USB_LP	USB_HP	ADC1	DMA1_CH7	DMA1_CH6	DMA1_CH5	DMA1_CH4	DMA1_CH3	DMA1_CH2	DMA1_CH1	EXTI4	EXTI3	EXTI2	EXTI1	EXTI0	RCC	FLASH	RTC_WKUP	TAMPER_STAMP	PVD	WWDG

Interrupt Clear Enable Register 1 (ICER1) *Address of ICER1 = Address of ICER0 + 4*

Clear Enable Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Interrupt Number																				44	43	42	41	40	39	38	37	36	35	34	33	32
																				1	0	0	0	0	0	0	0	0	0	0	0	0
																				TIM7	TIM6	USB_FS_WKUP	RTC_Alarm	EXTI15_10	USART3	USART2	USART1	SPI2	SPI1	I2C2_ER	I2C2_EV	I2C1_ER

TIM7_IRQn = 44

NVIC->ICER[1] = 1 << 12; // Disable Timer 7 interrupt

Disable/Enable Peripheral Interrupts

❑ For all peripheral interrupts: $IRQn \geq 0$

❑ Method 1:

❑ `NVIC_EnableIRQ (IRQn);` // Enable interrupt

❑ `NVIC_DisableIRQ (IRQn);` // Disable interrupt

❑ Method 2:

❑ Enable:

❑ `NVIC->ISER[ IRQn / 32] = 1 << (IRQn % 32);`

❑ Better solution:

❑ `NVIC->ISER[ IRQn >> 5] = 1 << (IRQn & 0x1F);`

❑ Disable:

❑ `NVIC->ICER[ IRQn >> 5] = 1 << (IRQn & 0x1F);`

Interrupt Priority

❓ Inverse Relationship:

❓ Lower priority value means higher urgency.

- ❓ Priority of Interrupt A = 5,
- ❓ Priority of Interrupt B = 2,
- ❓ B has a higher priority/urgency than A.

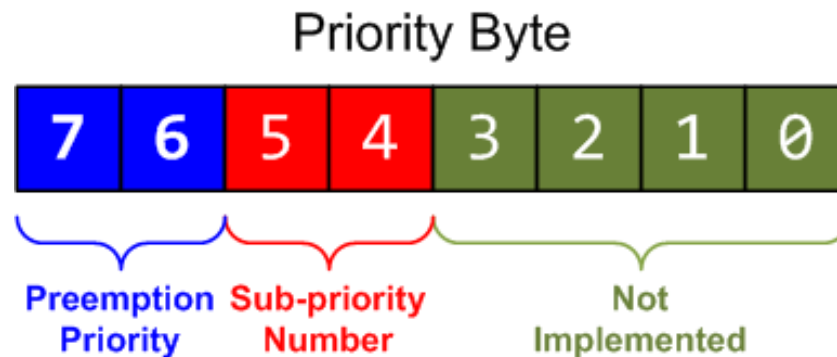
❓ Fixed priority for Reset, HardFault, and NMI.

Exception	IRQn	Priority
Reset	N/A	-3 (the highest)
Non-maskable Interrupt (NMI)	-14	-2 (2 nd highest)
Hard Fault	-13	-1

❓ Adjustable for all the other interrupts

Interrupt Priority

- ❑ Interrupt priority is configured by **Interrupt Priority Register (IP)**
- ❑ Each priority consists of two fields, including **preempt priority number** and **sub-priority number**.
 - ❑ The preempt priority number defines the priority for preemption.
 - ❑ The sub-priority number determines the order when multiple interrupts are pending with the same preempt priority number.

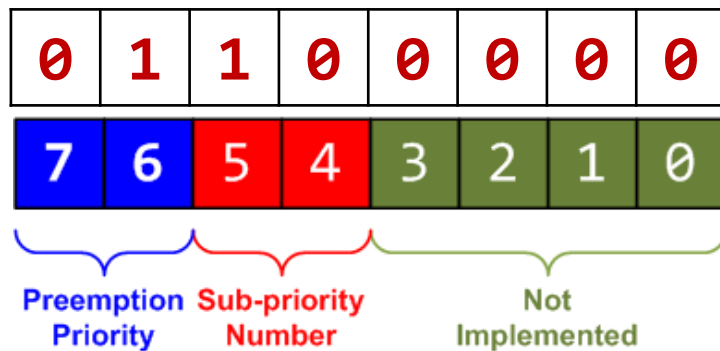


default setting

Interrupt Priority Levels

`NVIC_SetPriority(7, 6);`

`core_cm4.h` or `core_cm3.h`



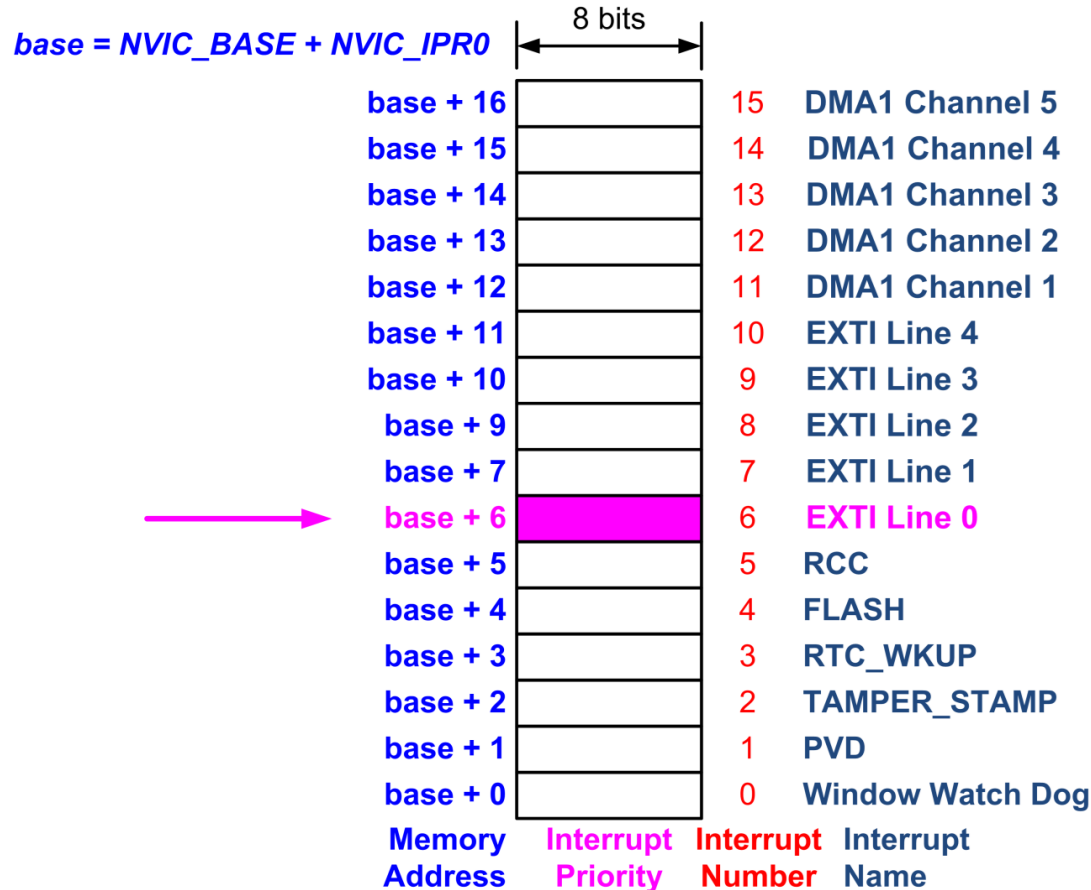
$IP = 0x60 = 96$

```
typedef struct {  
    ...  
    // Interrupt Priority Register  
    volatile uint8_t IP[240];  
    ...  
} NVIC_Type;
```

It is equivalent
to:

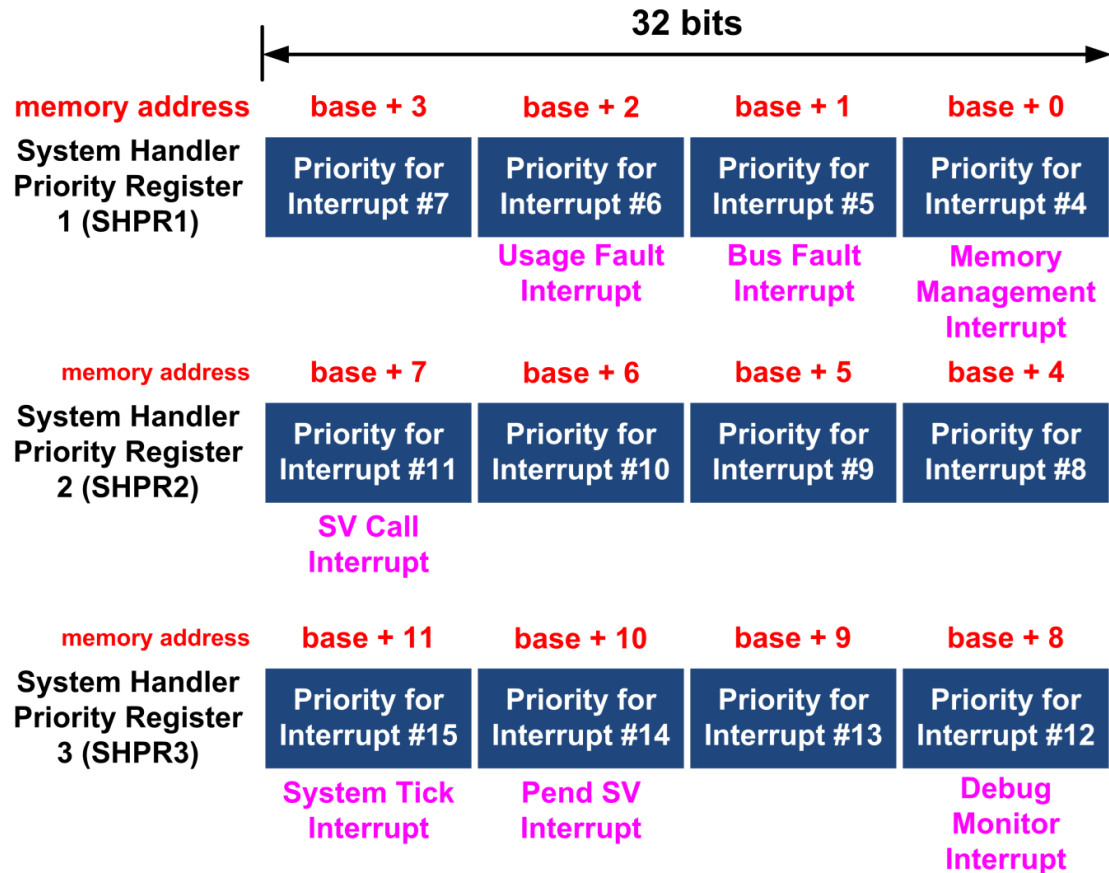
`NVIC->IP[7] = (6 << 4) & 0xff;`

Priority of Peripheral Interrupts



```
// Set the priority for EXTI 0 (Interrupt number 6)
NVIC->IP[6] = 0xF0;
```

Priority of System Interrupts



// Set the priority of a system interrupt IRQn
`SCB->SHP[(IRQn) & 0xF] - 4] = (priority << 4) & 0xFF;`

Masking Priority

- ❓ Disable all interrupts with less urgency
 - ❓ For critical code, only certain interrupts are allowed.
 - ❓ **BASEPRI**: Disable all interrupts of specific priority level or higher priority level

```
// Disable interrupts with priority value same or higher
__set_BASEPRI( 5 << 4 )

// Critical code
...

// Remove BASEPRI masking
__set_BASEPRI(0);
```

Exception-masking registers (PRIMASK, FAULTMASK and BASEPRI)

- ❓ **PRIMASK**: Used to disable all exceptions except Non-maskable interrupt (NMI) and hard fault.

- ❓ Write 1 to PRIMASK to disable all interrupts except NMI

```
MOV R0, #1  
MSR PRIMASK, R0
```

- ❓ Write 0 to PRIMASK to enable all interrupts

```
MOV R0, #0  
MSR PRIMASK, R0
```

- ❓ **FAULTMASK**: Like PRIMASK but change the current priority level to -1, so that even hard fault handler is blocked

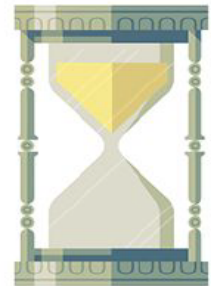
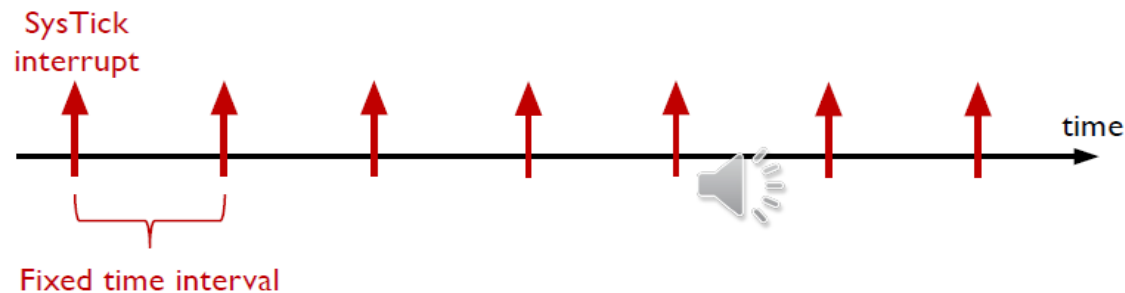
- ❓ **BASEPRI**: Disable interrupts only with priority lower than a certain level

- ❓ Example, disable all exceptions with priority level higher than 0x60

```
MOV R0, #0x60  
MSR BASEPRI, R0
```

System Timer (SysTick)

- Generate **SysTick interrupts** at a fixed time interval



- **Example Usages:**
 - Measuring time elapsed, such as time delay function
 - Executing tasks periodically, such as periodic polling, and OS CPU scheduling

System Timer (SysTick)

- System timer is a **standard** hardware component built into ARM Cortex-M.
- This hardware **periodically** forces the processor to execute the following ISR:

```
void SysTick_Handler(void){  
    ...  
}
```

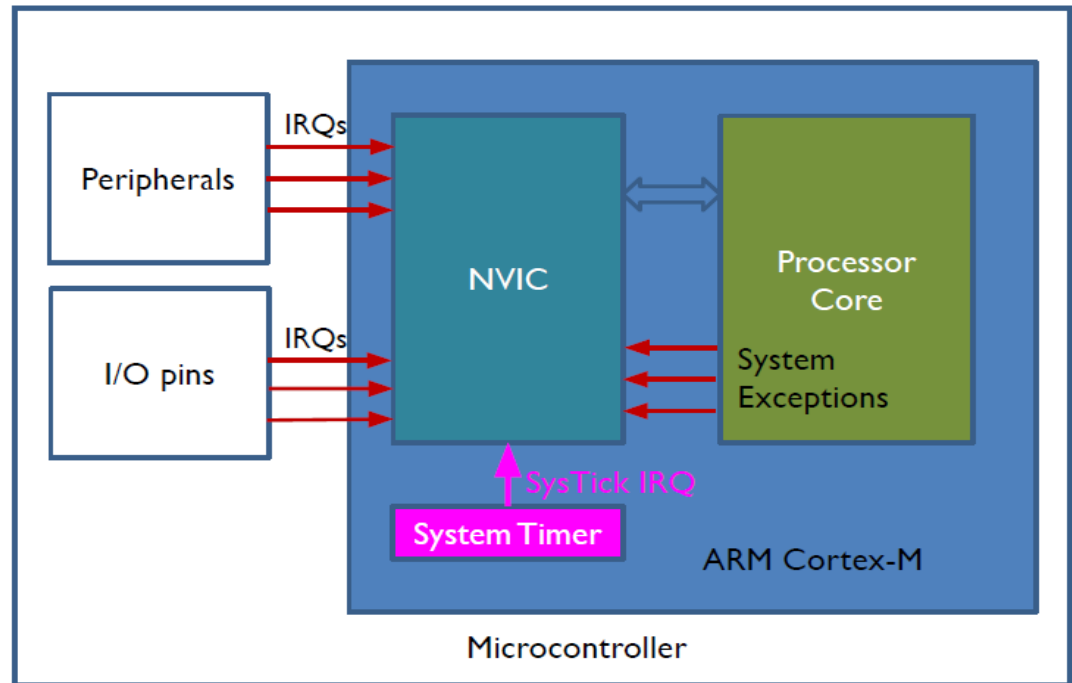
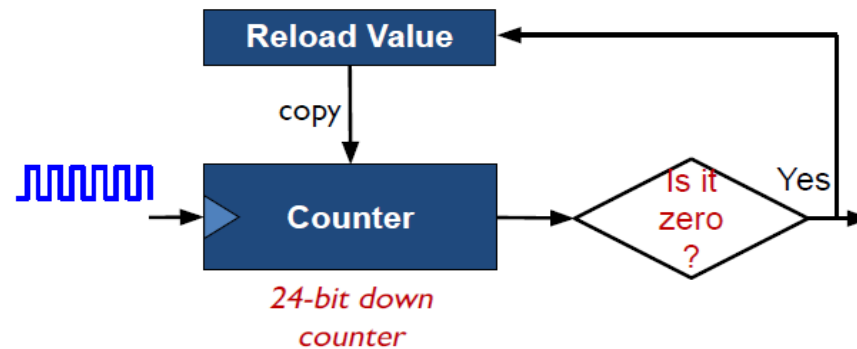


Diagram of System Timer (SysTick)



System Timer

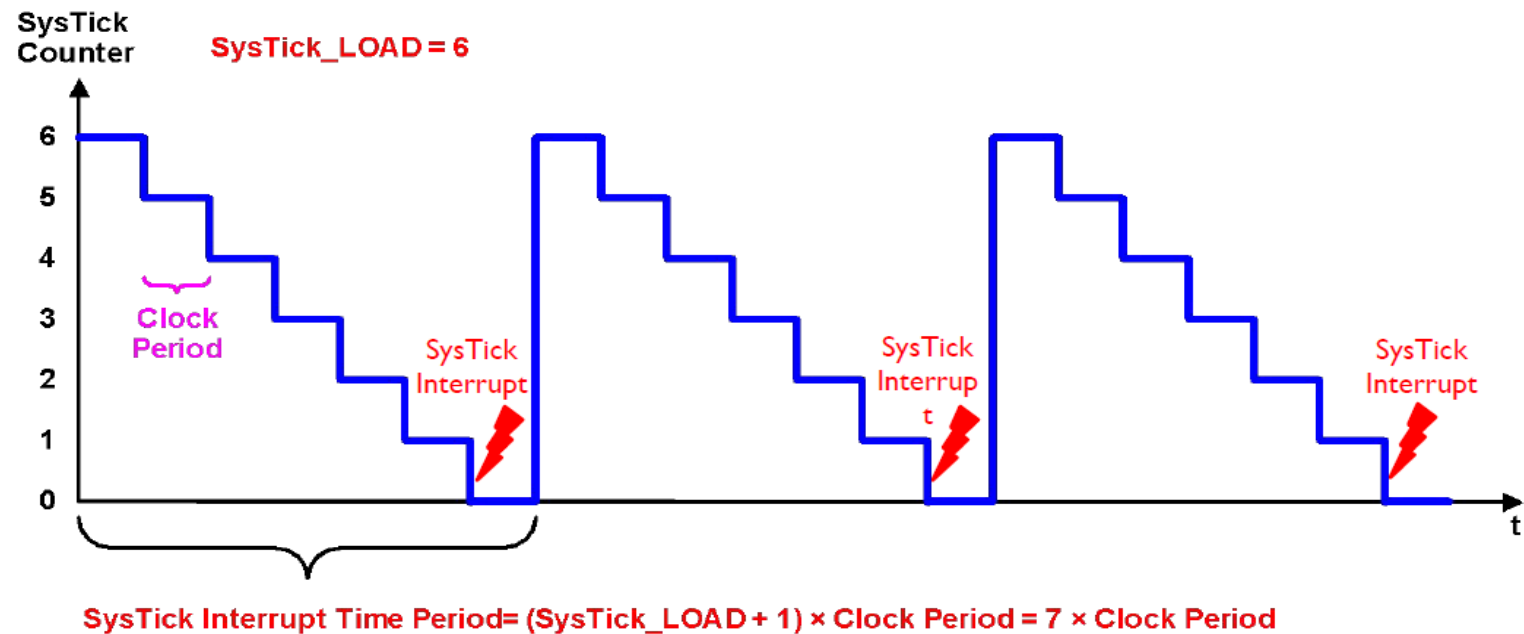


Diagram of System Timer (SysTick)

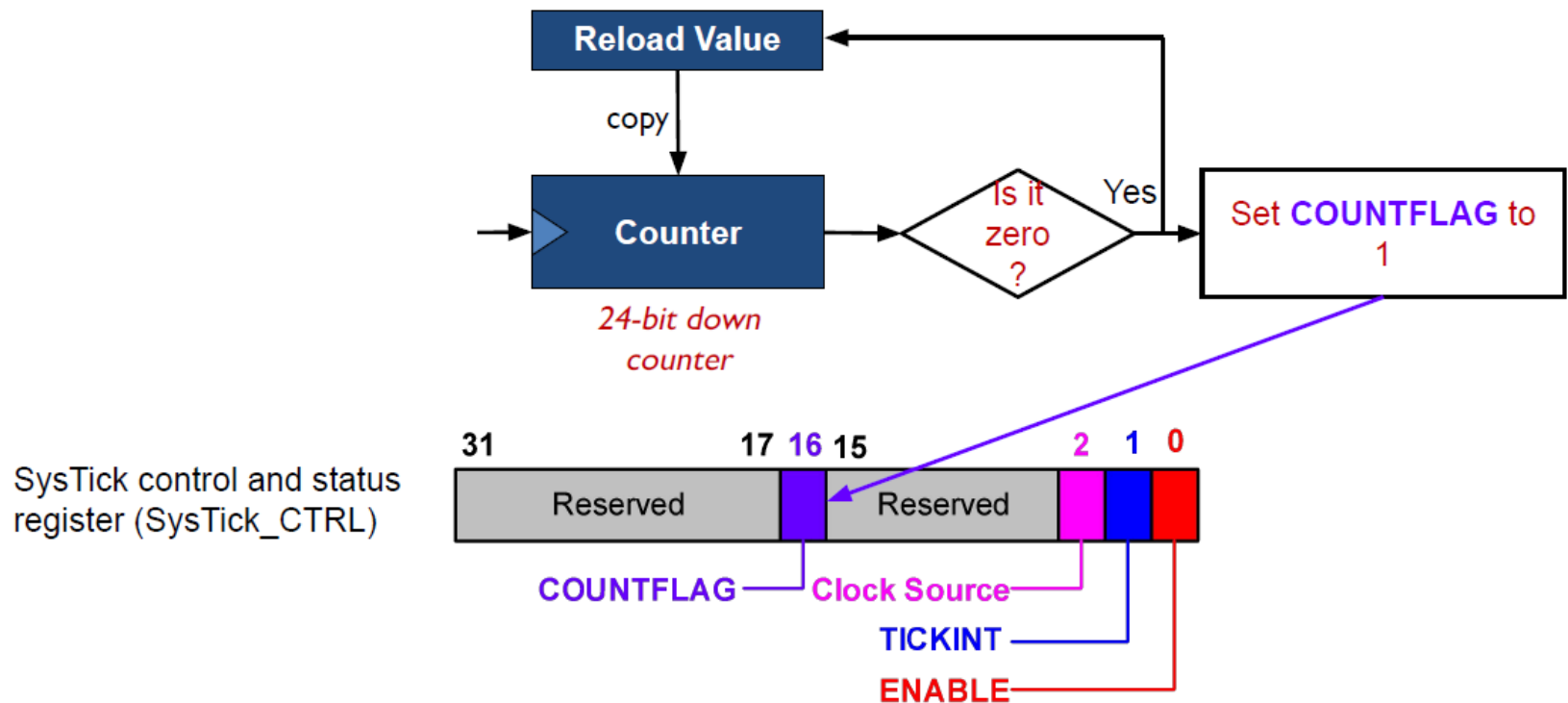
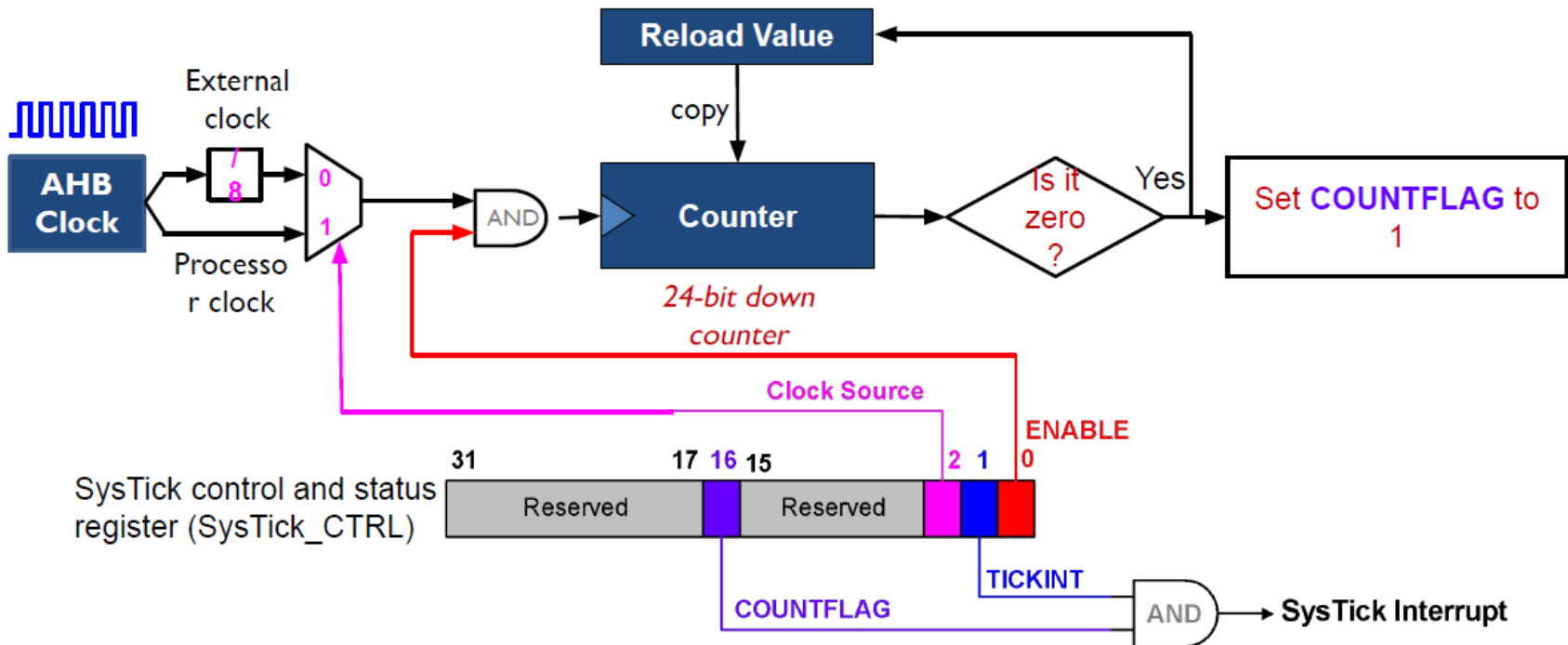
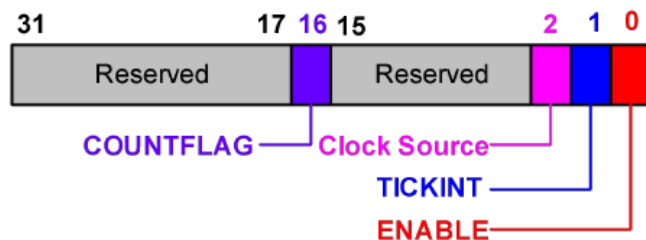


Diagram of System Timer (SysTick)



Registers of System Timer

SysTick control and status register (SysTick_CTRL)



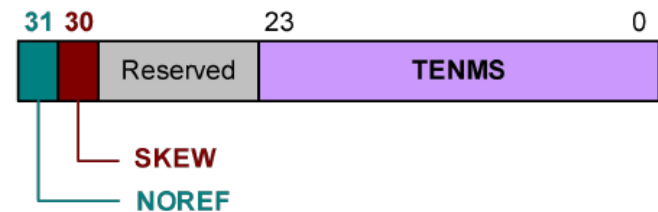
SysTick reload value register (SysTick_LOAD)



SysTick current value register (SysTick_VAL)



SysTick calibration register (SysTick_CALIB)



Registers of System Timer

SysTick reload value register (SysTick_LOAD)



- 24 bits, maximum value 0x00FF.FFFF (16,777,215)
- Counter counts down from RELOAD value to 0.
- Writing RELOAD to 0 disables SysTick, independently of TICKINT
- Time interval between two SysTick interrupts

$$\text{Interval} = (\text{RELOAD} + 1) \times \text{Source_Clock_Period}$$

- If 100 clock periods between two SysTick interrupts

$$\text{RELOAD} = 99$$

Registers of System Timer



- Reading it returns the current value of the counter
- When it transits from 1 to 0, it generates an interrupt
- Writing to SysTick_VAL clears the counter and COUNTFLAG to zero
 - Cause the counter to reload on the next timer clock
 - But, does not trigger an SysTick interrupt
- It has random value on reset.
 - Always clear it before enabling the timer

Registers of System Timer

SysTick calibration register (SysTick_CALIB)



- A read-only register
- TENMS (10 ms) holds the reload value, which will yield a 10ms period
- May not be implemented or may be defined differently by chip designers

Example Code

```
void SysTick_Initialize (uint32_t ticks) {  
    SysTick->CTRL = 0;           // Disable SysTick  
    SysTick->LOAD = ticks - 1;   // Set reload register  
    // Set interrupt priority of SysTick to least urgency (i.e., largest priority value)  
    NVIC_SetPriority (SysTick_IRQn, (1<<__NVIC_PRIO_BITS) - 1);  
    SysTick->VAL = 0;           // Reset the SysTick counter value  
    // Select processor clock: 1 = processor clock; 0 = external clock  
    SysTick->CTRL |= SysTick_CTRL_CLKSOURCE;  
    // Enables SysTick interrupt, 1 = Enable, 0 = Disable  
    SysTick->CTRL |= SysTick_CTRL_TICKINT;  
    // Enable SysTick  
    SysTick->CTRL |= SysTick_CTRL_ENABLE;  
}
```

Implementing Delay Function

```
volatile int32_t TimeDelay;

int main (void {
    SysTick_Initialize(1000); // Interrupt period = 1000 cycles
    Delay(100);               // Delay 100 ticks
    ...
}

void SysTick_Handler (void) { // SysTick interrupt service routine
    if (TimeDelay > 0)         // Prevent it from being negative
        TimeDelay--;          // TimeDelay is a global volatile variable
}

void Delay (uint32_t nTime) {
    // nTime: specifies the delay time length
    TimeDelay = nTime;         // TimeDelay must be declared as volatile
    while(TimeDelay != 0);     // Busy wait
}
```

Calculating Reload Value

- Suppose clock source = 80MHz
- Goal: SysTick Interval = 10ms
- What is RELOAD value?

$$\begin{aligned} \text{Reload} &= \frac{10 \text{ ms}}{\text{Clock Period}} - 1 \\ &= 10\text{ms} \times \text{Clock Frequency} - 1 \\ &= 10\text{ms} \times 80\text{MHz} - 1 \\ &= 10 \times 10^{-3} \times 80 \times 10^6 - 1 \\ &= 800000 - 1 \\ &= 799999 \end{aligned}$$

